



Kiln Defi Integrations

Security Review

Cantina Managed review by:

Noah Marconi, Lead Security Researcher

Xiaoming90, Security Researcher

August 27, 2024

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Low Risk	4
3.1.1	fToken Safeguards not Accessible Through the Vault Interface	4
3.1.2	Angle Saving Vault's maxDeposit and maxwithdraw do not comply with 4626 standard	5
3.2	Informational	7
3.2.1	Upgradeable external integrations	7
3.2.2	SparkLend vault unable to claim upcoming SPK airdrop	7

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	<i>Must fix as soon as possible (if already deployed).</i>
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Kiln is a staking platform you can use to stake directly, or whitelabel staking into your product. It enables users to stake crypto assets, manually or programmatically, while maintaining custody of your funds in your existing solution, such as Fireblocks, Copper, or Ledger.

From Aug 3rd to Aug 7th the Cantina team conducted a review of [defi-integrations](#) on commit hash [5de0e7c7](#). The team identified a total of **4** issues in the following risk categories:

- Critical Risk: 0
- High Risk: 0
- Medium Risk: 0
- Low Risk: 2
- Gas Optimizations: 0
- Informational: 2

3 Findings

3.1 Low Risk

3.1.1 fToken Safeguards not Accessible Through the Vault Interface

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: The fToken includes natspec comments cautioning about differences between its view functions and the amount of assets / shares related to a transaction. For both deposit and withdrawal there are non ERC4626 compliant functions the protocol recommends taking advantage of.

```
/// @inheritdoc ERC4626
/// @notice If `assets_` equals uint256.max then the whole fToken balance of `owner_` is withdrawn. This does
↳ not
///      consider withdrawal limit at Liquidity so best to check with `maxWithdraw()` before.
///      Note there might be tiny inaccuracies between requested `assets_` and actually received assets
↳ amount.
///      Recommended to use `withdraw()` with a `minAmountOut_` param instead to set acceptable limit.
/// @return shares_ burned shares
function withdraw(
    uint256 assets_,
    address receiver_,
    address owner_
) public virtual override nonReentrant returns (uint256 shares_) {
    if (assets_ == type(uint256).max) {
        assets_ = previewRedeem(balanceOf(owner_));
    }
    shares_ = _executeWithdraw(assets_, receiver_, owner_);

    if (msg.sender != owner_) {
        _spendAllowance(owner_, msg.sender, shares_);
    }
}

/// @notice same as {fToken-withdraw} but with an additional setting for maximum shares burned.
/// reverts with `fToken__MaxAmount()` if `maxSharesBurn_` of shares burned is surpassed.
function withdraw(
    uint256 assets_,
    address receiver_,
    address owner_,
    uint256 maxSharesBurn_
) external returns (uint256 shares_) {
    shares_ = withdraw(assets_, receiver_, owner_);
    _revertIfAboveMaxAmount(shares_, maxSharesBurn_);
}
```

Related, the convenience of passing in `assets_ == type(uint256).max` to "withdraw all" is not reachable through the Kiln Vault Interface.

Recommendation: Consider if an interface change, or larger refactor, is desirable to handle asset to share price movement between the time a transaction is constructed and included in a block.

Kiln: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 Angle Saving Vault's `maxDeposit` and `maxWithdraw` do not comply with 4626 standard

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: Following is the Angle Saving Vault on Mainnet:

- stUSD (0x0022228a2cc5E7eF0274A7Baa600d44da5aB5776).
- stEUR (0x004626A008B1aCdC4c74ab51644093b155e59A23).

- **Instance 1:** `maxDeposit`

Following is the specification of the `maxDeposit` function as per EIP-4626:

Maximum amount of the underlying asset that can be deposited into the Vault for the receiver, through a deposit call.

MUST return the maximum amount of assets deposit would allow to be deposited for receiver and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary). This assumes that the user has infinite assets, i.e. MUST NOT rely on `balanceOf` of asset.

MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0.

MUST return $2^{256} - 1$ if there is no limit on the maximum amount of assets that may be deposited.

MUST NOT revert.

Note that the specification stated that if deposits are entirely disabled (even temporarily), the `maxDeposit` function must return 0.

The following is the implementation of the Angle Saving Vault's `deposit` function. This function is pausable via the `whenNotPaused` modifier. Thus, when the contract is paused, the vault's deposit is disabled.

```
function deposit(uint256 assets, address receiver) public override whenNotPaused returns (uint256
↪ shares) {
    uint256 newTotalAssets = _accrue();
    shares = _convertToShares(assets, newTotalAssets, MathUpgradeable.Rounding.Down);
    _deposit(_msgSender(), receiver, assets, shares);
}
```

The following is the implementation of the Angle Saving Vault's `maxDeposit` function. It was observed that even if the contract is paused and the deposit is disabled, the function continues to return `type(uint256).max` instead of zero. This means that the Angle Saving Vault's `maxDeposit` function does not comply with the EIP specification.

```
function maxDeposit(address) public view virtual override returns (uint256) {
    return _isVaultCollateralized() ? type(uint256).max : 0;
}
```

Since Kiln's Vault `maxDeposit` function internally relies on Angle Saving Vault's `maxDeposit` function, it will inherit the same issue. When Angle Saving Vault's `deposit` is disabled, Kiln's vault `deposit` is also disabled, as an attempt to deposit will result in a revert. However, Kiln's vault `maxDeposit` will incorrectly return `type(uint256).max` instead of zero in such a scenario.

```
File: AngleSavingConnector.sol
56:     function maxDeposit(IERC20) external view override returns (uint256) {
57:         return stakingVault.maxDeposit(msg.sender);
58:     }
```

- **Instance 2:** `maxWithdraw`

Following is the specification of the `maxWithdraw` function as per EIP-4626:

Maximum amount of the underlying asset that can be withdrawn from the `owner` balance in the Vault, through a `withdraw` call.

MUST return the maximum amount of assets that could be transferred from `owner` through `withdraw` and not cause a revert, which MUST NOT be higher than the actual maximum that would be accepted (it should underestimate if necessary).

MUST factor in both global and user-specific limits, like if withdrawals are entirely disabled (even temporarily) it MUST return 0.

MUST NOT revert.

Note that the specification stated that if withdrawals are entirely disabled (even temporarily), the `maxWithdraw` function must return 0.

The following is the implementation of the Angle Saving Vault's `withdraw` function. This function is pausable via the `whenNotPaused` modifier. Thus, when the contract is paused, the vault's `withdraw` is disabled.

```
function withdraw(
    uint256 assets,
    address receiver,
    address owner
) public override whenNotPaused returns (uint256 shares) {
    uint256 newTotalAssets = _accrue();
    shares = _convertToShares(assets, newTotalAssets, MathUpgradeable.Rounding.Up);
    _withdraw(_msgSender(), receiver, owner, assets, shares);
}
```

The following is the implementation of the Angle Saving Vault's `maxWithdraw` function. It was observed that even if the contract is paused and the withdrawal is disabled, the function continues to return `type(uint256).max` instead of zero. This means that the Angle Saving Vault's `maxWithdraw` function does not comply with the EIP specification.

```
function maxWithdraw(address owner) public view virtual override returns (uint256) {
    return _convertToAssets(balanceOf(owner), MathUpgradeable.Rounding.Down);
}
```

Since Kiln's Vault `maxWithdraw` function internally relies on Angle Saving Vault's `maxWithdraw` function, it will inherit the same issue. When Angle Saving Vault's `withdraw` is disabled, Kiln's vault `withdraw` is also disabled, as an attempt to withdraw will result in a revert. However, Kiln's vault `maxWithdraw` will incorrectly return `type(uint256).max` instead of zero in such a scenario.

```
File: AngleSavingConnector.sol
61:     function maxWithdraw(IERC20) external view override returns (uint256) {
62:         return stakingVault.maxWithdraw(msg.sender);
63:     }
```

Recommendation: Consider one of the following two approaches:

1. Update Kiln's Vault `maxDeposit` and `maxWithdraw` functions to return zero if the Angle Saving Vault is paused. This will ensure that Kiln's vault complies with the EIP standard even if Angle Saving Vault's `maxDeposit` and `maxWithdraw` functions do not (**Recommended**).
2. Keep the existing implementation to ensure consistency between Kiln and Angle Saving's `maxDeposit`/`maxWithdraw` implementations, and document the non-compliance accordingly to ensure that the integrators are aware of such an issue.

Kiln: Fixed in [PR 168](#).

3.2 Informational

3.2.1 Upgradeable external integrations

Severity: Informational

Context: [AngleSavingConnector.sol#L25](#)

Description: A number of the pools integrated with are upgradeable. An erroneous or unanticipated upgraded interface change can lead to locked funds or other issues. Noting that the Kiln vaults are also upgradeable, there is no permanent DOS vector.

Recommendation: Nonetheless, monitoring is recommended for each integration to anticipate issues with upgrades. Consider monitoring + pausing when any external integration experiences an upgrade. Kiln team has noted, monitoring is currently in place.

Kiln: Acknowledged.

Cantina Managed: Acknowledged.

3.2.2 SparkLend vault unable to claim upcoming SPK airdrop

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: As part of the [Maker Endgame plan](#), depositors of certain assets will be eligible for the upcoming SPK airdrop. However, the claim function is not implemented in the current implementation of the Spark Lend connector and will revert if executed.

```
File: SparkLendConnector.sol
099:     /// @inheritdoc IConnector
100:     function claim(IERC20, IERC20, bytes calldata) external pure override {
101:         revert NothingToClaim();
102:     }
```

Recommendation: Consider one of the following two approaches:

- Implement a claim function that accepts arbitrary execution data. Robust access control must be put in place to guard against potential abuse of such a function.
- Monitor the Maker Endgame plan closely, and upgrade the SparkLendConnector contract to handle the SPK airdrop when the tokens are distributed.

Kiln: Acknowledged, we will upgrade the connector with the claim function once needed.

Cantina Managed: Acknowledged.